

# El Algoritmo de Shor

Factorización de enteros en tiempo polinómico cuántico

---

Juan Ignacio Fernandez Dinardo    Francisco Marcos Ferrutti

Junio 2026

Computación Cuántica · Ingeniería Informática · ITBA

# Motivación y RSA

---

1. Motivación y por qué factorizar rompe RSA
2. El problema: factorización como *order-finding* (hallar el período)
3. Prerrequisitos sobre el número  $N$
4. El algoritmo: parte clásica + parte cuántica
5. Por qué el período permite factorizar — la cadena algebraica completa
6. La QFT y el circuito cuántico
7. Ejemplo resuelto con  $N = 15$ : un fallo ( $k = 14$ ) y un éxito ( $k = 7$ )
8. Complejidad: Shor vs. el mejor clásico (GNFS)
9. Implicancias, hardware actual y criptografía post-cuántica

## Motivación: por qué nos importa

- Gran parte de la seguridad de Internet (TLS, firmas, certificados) se apoya en que **factorizar un entero grande es difícil** para una computadora clásica.
- “Difícil”: el mejor algoritmo clásico conocido (GNFS) tarda **tiempo sub-exponencial** — para RSA-2048, fuera del alcance práctico.
- En 1994, Peter Shor mostró un algoritmo **cuántico** que factoriza en **tiempo polinómico**.  
Eso convierte un problema “imposible” en uno “tratable”.
- Consecuencia: con hardware cuántico suficiente, **RSA y los esquemas basados en logaritmo discreto (DH, DSA, ECDSA)** dejarían de ser seguros.

## Generación de claves (textbook RSA)

- Se eligen dos primos  $p, q$  y se define el módulo  $N = p \cdot q$ .
- Se calcula  $\varphi(N) = (p - 1)(q - 1)$ .
- Se elige  $e$  con  $\gcd(e, \varphi(N)) = 1$  y se obtiene  $d$  tal que  $e \cdot d \equiv 1 \pmod{\varphi(N)}$ .
- Clave pública:  $(N, e)$     Clave privada:  $(N, d)$ .

Cifrado:

$$c \equiv m^e \pmod{N}$$

Descifrado:

$$m \equiv c^d \pmod{N}$$

Lo público:  $(N, e)$ .    El secreto efectivo:  $d$ .     $d$  se obtiene de  $\varphi(N)$ , y  $\varphi(N)$  de  $p$  y  $q$ .

## Por qué factorizar $N$ rompe RSA



- El atacante conoce  $(N, e)$  (públicos).
- Si logra **factorizar**  $N = p \cdot q$ , calcula  $\varphi(N)$  trivialmente.
- Con  $\varphi(N)$  y  $e$ , obtiene  $d$  con el **algoritmo de Euclides extendido** (eficiente).
- Con  $d$  descifra cualquier mensaje y firma como el dueño de la clave.

La seguridad de RSA se apoya por completo en que nadie pueda factorizar  $N$ . Shor ataca ese único punto.

## El problema y prerrequisitos

---

## El problema real: factorización $\equiv$ order-finding

Shor **no** factoriza por fuerza bruta. Reduce la factorización a:

**hallar el período (orden) de una función.**

### Definición

Dado  $N$  y un  $k$  coprimo con  $N$ , se define  $f(x) = k^x \bmod N$ . Esta función es **periódica**: existe mínimo  $r > 0$  tal que  $f(x + r) = f(x) \forall x$ , es decir  $k^r \equiv 1 \pmod{N}$ . A  $r$  se lo llama el **orden de  $k$  módulo  $N$** .

### Idea central

Conocer  $r$  permite, con álgebra elemental + GCD, recuperar factores de  $N$ .

Hallar  $r$  es **lo único que necesita la computadora cuántica**.

## Prerrequisitos sobre el input $N$

Antes de correr Shor, se descartan clásicamente los casos triviales.  $N$  debe cumplir:

1.  $N$  no es primo.

Si fuera primo, no hay nada que factorizar (test de primalidad clásico y eficiente, p. ej. Miller-Rabin).

2.  $N$  no es par.

Si lo fuera, 2 ya es un factor (se chequea mirando el último bit).

3.  $N$  no es una potencia perfecta  $n^x$  (con  $n, x \in \mathbb{Z}$ ,  $x \geq 2$ ).

Si lo fuera, la raíz  $n$  es un factor, hallable con métodos clásicos eficientes (probar raíces  $x$ -ésimas para  $x$  hasta  $\log_2 N$ ).

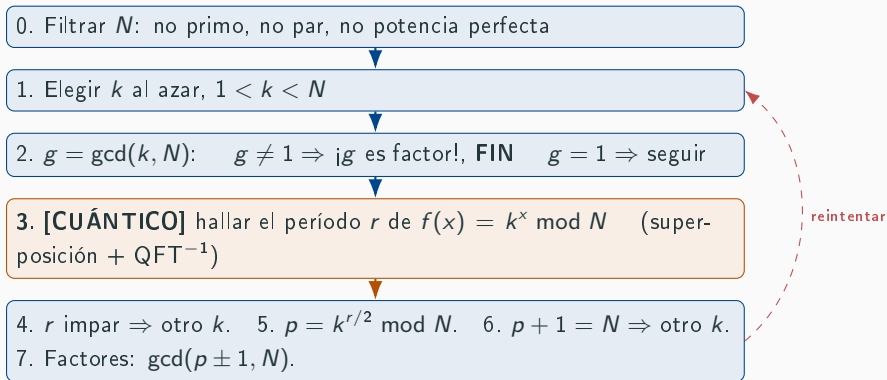
Si  $N$  pasa estos filtros, es un **compuesto impar con al menos dos factores primos distintos** — el caso para el que Shor está diseñado.

## El corazón: el período de $f(x) = k^x \bmod N$

- $f(x) = k^x \bmod N$  es **periódica** con período  $r = \text{orden de } k \text{ módulo } N$ .
- Hallar  $r$  es el **único paso que requiere la computadora cuántica**.
- Clásicamente no hay método eficiente: el período puede ser astronómicamente grande para  $N$  de miles de bits.
- La cuántica evalúa  $f$  en **superposición sobre todos los  $x$**  y usa interferencia (QFT) para extraer la periodicidad.

Todos los demás pasos (GCD, chequeos, derivar factores) son **clásicos y polinómicos**. La ventaja cuántica vive enteramente acá.

# Esqueleto del algoritmo: clásico vs. cuántico



## La parte clásica, en detalle

1. **Elegir  $k$**  uniformemente al azar en  $1 < k < N$ .
2. **GCD inicial:**  $g = \gcd(k, N)$ .
  - Si  $g \neq 1$ :  $g$  es factor no trivial  $\rightarrow$  listo (sin cuántica).
  - Si  $g = 1$ :  $k$  es coprimo con  $N \rightarrow$  buscar su orden.
3. *(cuántico: obtener  $r$ )*
4. **Paridad:** si  $r$  es impar  $\rightarrow$  no sirve, elegir otro  $k$ .
5. **Calcular  $p = k^{r/2} \bmod N$ .**  
(por exponenciación binaria: rápido en clásico)
6. **Chequeo de fallo:** si  $p = N - 1$  (equivale a  $k^{r/2} \equiv -1$ )  $\rightarrow$  no sirve, otro  $k$ .
7. **Extraer factores:**  $\gcd(p - 1, N)$  y  $\gcd(p + 1, N)$  dan factores no triviales.

**Nota de notación:** en lo que sigue,  $p = k^{r/2} \bmod N$  es un **residuo**, *no* un primo. No confundir con los primos  $p, q$  de RSA.

# GCD y Euclides: honestidad sobre su uso

¿Cómo se calcula  $\gcd(a, N)$ ? Con el algoritmo de Euclides:

$\gcd(a, b)$  : mientras  $b \neq 0$  :  $a, b \leftarrow b, a \bmod b$ ; devolver  $a$ .

Idea:  $\gcd(a, b) = \gcd(b, a \bmod b)$ , reduciendo hasta que un término sea 0.

¿Se usa en la práctica?

Sí. Euclides es eficiente ( $O((\log N)^2)$  o mejor) y es exactamente lo que se ejecuta para los GCDs al implementar Shor en, por ejemplo, Qiskit.

¿Qué *no* se hace clásicamente?

El paso de **hallar el período**  $r$ : no existe método clásico eficiente conocido. Por eso va a la computadora cuántica.

Los GCDs y el resto del post-procesamiento son baratos. La dificultad está concentrada, otra vez, en **hallar el período**.

Por qué el período permite  
factorizar

---

## Por qué el período permite factorizar (1/5): punto de partida

Partimos de la definición de  $r$ :

$$k^r \equiv 1 \pmod{N} \iff k^r - 1 \equiv 0 \pmod{N} \iff N \mid (k^r - 1).$$

Es decir:  $N$  **divide a**  $k^r - 1$ .

A partir de ahí vamos a demostrar, encadenadas, cuatro cosas:

- (a) por qué  $r$  debe ser **par**,
- (b) por qué exigimos  $k^{r/2} + 1 \not\equiv 0 \pmod{N}$ ,
- (c) por qué  $\gcd(k^{r/2} \pm 1, N)$  da **factores no triviales**,
- (d) por qué todo esto ocurre con **alta probabilidad**.

## (a) Por qué $r$ debe ser par

Tenemos  $N \mid (k^r - 1)$ . Si  $r$  es **par**, definimos  $p_0 = r/2$  entero y factorizamos como diferencia de cuadrados:

$$k^r - 1 = (k^{r/2})^2 - 1 = \underbrace{(k^{r/2} - 1)}_{\text{factor A}} \underbrace{(k^{r/2} + 1)}_{\text{factor B}}.$$

Esta factorización **abre la puerta**:  $N$  divide al producto de dos números más manejables.

Si  $r$  es **impar**:

- $r/2$  no es entero  $\Rightarrow$  esa factorización no existe en  $\mathbb{Z}$ .
- **Se descarta este  $k$  y se reintenta** con otro.

**Notación:** definimos  $p = k^{r/2} \bmod N$  (el *residuo*). Este  $p$  **no** es un primo.

## (b) Por qué se exige $k^{r/2} + 1 \not\equiv 0 \pmod{N}$

De (a):  $N \mid (k^{r/2} - 1)(k^{r/2} + 1)$ .

Para que esto sea **útil**,  $N$  tiene que **repartir** sus primos entre los dos factores. El intento **falla** si  $N$  divide enteramente a uno solo:

### Subcaso 1 — imposible

$N \mid (k^{r/2} - 1)$  implica  $k^{r/2} \equiv 1 \pmod{N}$ .

Pero eso contradiría que  $r$  es el período *mínimo*.  $\Rightarrow$  **no puede ocurrir**.

### Subcaso 2 — modo de fallo

$N \mid (k^{r/2} + 1)$  implica  $k^{r/2} \equiv -1 \pmod{N}$ ,

es decir  $p = N - 1$ .

$\Rightarrow$  **descartar  $k$ , reintentar**.

Es el fallo del *caso feo*  $k=14$ .

En ambos fallos,  $\gcd(k^{r/2} \pm 1, N)$  resultaría trivial (1 o  $N$ ).

### (c) Por qué $\gcd(k^{r/2} \pm 1, N)$ da factores no triviales

Tras el chequeo (b):  $N$  **no** divide enteramente a ninguno de los factores  $k^{r/2} - 1$ ,  $k^{r/2} + 1$ . Pero sí divide al **producto**.

Como  $N = q_1 \cdot q_2 \cdots$ , y ningún paréntesis se lleva todos sus primos: algún primo divide a  $k^{r/2} - 1$  y otro a  $k^{r/2} + 1$ .

$$\gcd(k^{r/2} - 1, N) \quad \text{y} \quad \gcd(k^{r/2} + 1, N)$$

devuelven **divisores propios** de  $N$  (ni 1 ni  $N$ ). Para  $N = 15$ : **3 y 5**.

Es el **GCD** —no la resta directa— lo que aísla el factor. Como Euclides es eficiente, **este paso es clásico y barato**.

## (d) “Alta probabilidad”: la cota cuantitativa

### Theorem (Nielsen & Chuang, Teorema A4.13)

Sea  $N$  compuesto impar con  $m$  factores primos distintos. Si  $k$  se elige uniformemente al azar entre los coprimos con  $N$ :

$$\Pr[r \text{ par y } k^{r/2} \not\equiv -1 \pmod{N}] \geq 1 - \frac{1}{2^{m-1}}.$$

- $m \geq 2$  (tras los filtros)  $\Rightarrow$  éxito  $\geq 1/2$  por intento.
- Más primos distintos  $\Rightarrow$  cota  $\rightarrow 1$ .
- Reintentos esperados:  $\leq 2$ .

#### Fuente

El paper *no* incluye esta cota (y tiene una errata).

Se cita Nielsen & Chuang como fuente estándar.

## La parte cuántica: QFT y circuitos

---

## La parte cuántica: hallar el período

Dos registros:

- **Conteo** ( $t$  qubits,  $|0\rangle^{\otimes t}$ ): codifica la fase que revela  $r$ . Se elige  $2^t \geq N^2$ .
- **Trabajo** ( $n = \lceil \log_2 N \rceil$  qubits,  $|1\rangle$ ): almacena  $k^x \bmod N$ .

Pasos:

1. Hadamards en conteo  $\rightarrow$  superposición uniforme.
2. Exponenciación modular controlada:  $|x\rangle |1\rangle \rightarrow |x\rangle |k^x \bmod N\rangle$ .
3. **QFT**<sup>-1</sup> sobre el registro de conteo.
4. Medir conteo  $\rightarrow c \approx s \cdot 2^t / r$ .
5. **Fracciones continuas** (clásico)  $\rightarrow r$ .

# La QFT: qué es

La **Transformada de Fourier Cuántica** sobre  $n$  qubits ( $M = 2^n$  estados base) actúa:

$$\text{QFT } |j\rangle = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{2\pi i jk/M} |k\rangle.$$

- Mapea  $|j\rangle$  a una **superposición con fases que giran** a “frecuencia” proporcional a  $j$ .
- Sobre una superposición periódica, las fases **interfieren constructivamente** en los múltiplos de  $M/r$  y **destructivamente** en el resto.
- Al medir, los valores probables están en  $c \approx s \cdot M/r \rightarrow$  de ahí se extrae  $r$ .

## Implementación eficiente

Se construye con **Hadamards** y **rotaciones de fase controladas**  $R_d = \text{diag}(1, e^{2\pi i/2^d})$ . Usa  $O(n^2)$  compuertas.

## QFT vs. DFT/FFT clásica

|                 | DFT/FFT (clásica)                          | QFT (cuántica)                           |
|-----------------|--------------------------------------------|------------------------------------------|
| Sobre qué actúa | Vector de $M$ amplitudes explícitas        | Amplitudes de $n = \log_2 M$ qubits      |
| Costo           | FFT: $O(M \log M) = O(2^n \cdot n)$        | $O(n^2) = O((\log M)^2)$ compuertas      |
| Salida          | Todas las $M$ componentes, <b>legibles</b> | Una sola muestra (probabilística)        |
| Acceso          | Lectura completa del espectro              | No se puede leer todo (medición colapsa) |

### La diferencia clave

La QFT es **exponencialmente más rápida** en operaciones. . .

. . . pero **no entrega el espectro completo**: solo permite *muestrear* una salida.

El arte de Shor: diseñar el circuito para que **lo que se muestrea codifique el período**  $r$ .

## Estimación de fase: el motor del order-finding

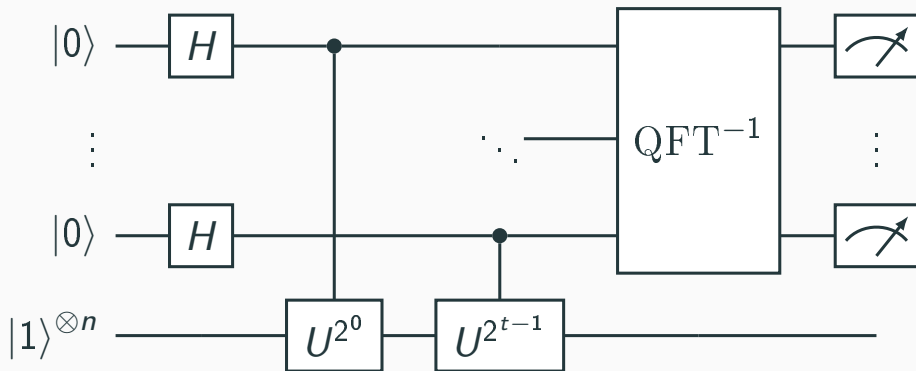
El subprograma de período es una **estimación de fase** del operador  $U$ :

$$U|y\rangle = |k \cdot y \bmod N\rangle.$$

- Los autovectores de  $U$  tienen autovalores  $e^{2\pi i s/r}$  ( $s = 0, \dots, r-1$ ).
- El estado inicial  $|1\rangle$  del registro de trabajo es **superposición de esos autovectores**.
- Aplicando  $U$  controlada  $2^j$  veces desde cada qubit de conteo, se “imprime” la fase  $s/r$  en el registro de conteo.
- La **QFT inversa** convierte esa fase en un entero medible  $c \approx s \cdot 2^t/r$ .

$$k^x \bmod N \longrightarrow \text{periodicidad } r \longrightarrow \text{fases } s/r \longrightarrow \text{QFT}^{-1} \longrightarrow \text{medir } c \approx s \cdot 2^t/r.$$

# El circuito cuántico completo



1.  $H \rightarrow$  superposición.
2. Exp. modular controlada:  $|x\rangle |1\rangle \rightarrow |x\rangle |k^x \bmod N\rangle$ .
3.  $QFT^{-1}$ .
4. Medir conteo  $\rightarrow c$ . (Trabajo: no se mide.)

## El bloque costoso: exponenciación modular controlada

$$U : |y\rangle \rightarrow |k \cdot y \bmod N\rangle \quad \text{encadenando:} \quad |x\rangle |1\rangle \rightarrow |x\rangle |k^x \bmod N\rangle$$

- Se trata como un **bloque abstracto** en el circuito (no desarrollamos su construcción interna a nivel compuertas).
- Internamente: sumadores modulares, multiplicadores y exponenciación por cuadrados controlada.
- **Es el bloque más costoso del algoritmo:** domina el conteo de compuertas y de qubits auxiliares (ancillas). La complejidad total de Shor está gobernada por este paso.

En implementaciones reales (Qiskit y similares) esta es la parte que más recursos consume y la más optimizada. La QFT, en comparación, es barata.

## De la medición al período: fracciones continuas

La medición da  $c$  con  $c/2^t \approx s/r$  (para algún  $s$  desconocido). Para recuperar  $r$  se usa el **desarrollo en fracciones continuas** de  $c/2^t$ : sus **convergentes** dan la mejor aproximación racional con denominador acotado, y ese denominador es (un divisor de)  $r$ .

$$\frac{c}{2^t} = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \cdots}} \longrightarrow \text{convergentes } \frac{p_i}{q_i} \longrightarrow r = q_i.$$

### Importante para $N = 15$

Todos los  $k$  coprimos tienen  $r \in \{2, 4\}$  (potencias de 2). Las fracciones  $s/r$  salen **exactas** y las fracciones continuas son triviales. **No hay “caso feo” de fracciones continuas con  $N = 15$ .** Un caso no trivial requeriría  $N$  mayor (p.ej.  $N = 21$ ,  $r = 6$ ).

**Ejemplo resuelto:  $N = 15$**

---

## Ejemplo $N = 15$ : preparación

### Filtros (prerrequisitos):

- ¿Primo? No ( $15 = 3 \times 5$ ).    ¿Par? No.    ¿Potencia perfecta? No.
- ✓ 15 es compuesto impar con 2 primos distintos. **Apto para Shor.**

**Registros:**  $n = \lceil \log_2 15 \rceil = 4$  qubits (trabajo);  $2^t \geq 225 \Rightarrow t = 8$  qubits ( $2^8 = 256$ ).

### Notación importante

$p = k^{r/2} \bmod 15$  es un **residuo**, *no* un primo. Los factores buscados son 3 y 5.

Dos casos: **Caso feo (fallo):**  $k = 14$ .    **Caso feliz (éxito):**  $k = 7$ .

## Caso feo (fallo): $k = 14$

### 1. Coprimalidad

$\gcd(14, 15) = 1 \checkmark$  (coprimo, seguimos)

### 2. Período $f(x) = 14^x \bmod 15$

|        |   |    |   |    |   |     |
|--------|---|----|---|----|---|-----|
| $x$    | 0 | 1  | 2 | 3  | 4 | ... |
| $14^x$ | 1 | 14 | 1 | 14 | 1 | ... |

 $\Rightarrow r = 2$  (par  $\checkmark$ )

### 3. Calcular $p = 14^{r/2} \bmod 15 = 14^1 \bmod 15 = 14$

### 4. Chequeo de fallo: $p + 1 = 15 = N$ ¡FALLA!

$14^{r/2} \equiv -1 \pmod{15}$ :  $N$  divide por completo a  $(p + 1)$ .

Si igual calculáramos:  $\gcd(13, 15) = 1$     $\gcd(15, 15) = 15$     $\rightarrow$  triviales.

**Conclusión:**  $k = 14$  no factoriza. Reintentar con otro  $k$ .

## Caso feliz ( $k = 7$ ): parte clásica

1.  $\gcd(7, 15) = 1 \checkmark$

2. **Período**  $f(x) = 7^x \bmod 15$  (lo calculamos en la cuántica)

|       |   |   |   |    |   |   |   |     |
|-------|---|---|---|----|---|---|---|-----|
| $x$   | 0 | 1 | 2 | 3  | 4 | 5 | 6 | ... |
| $7^x$ | 1 | 7 | 4 | 13 | 1 | 7 | 4 | ... |

$\Rightarrow r = 4$  (par  $\checkmark$ )

3.  $p = 7^{r/2} \bmod 15 = 7^2 \bmod 15 = 49 \bmod 15 = 4$

4. **Chequeo:**  $p + 1 = 5 \neq 15 \checkmark$  ( $7^2 \not\equiv -1$ )

### 5. Extraer factores con GCD

$$\gcd(p - 1, 15) = \gcd(3, 15) = 3 \qquad \gcd(p + 1, 15) = \gcd(5, 15) = 5$$

**Factores de 15: 3 y 5.  $\checkmark\checkmark$**

## Caso $k = 7$ — parte cuántica: la periodicidad

$f(x) = 7^x \bmod 15$ , evaluada en superposición:

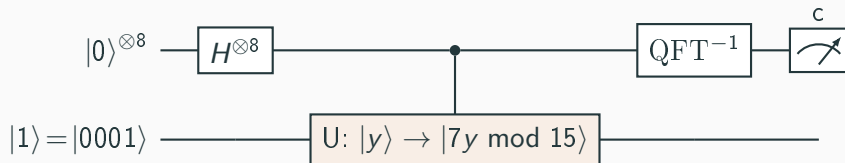
| $x$            | 0 | 1 | 2 | 3  | 4 | 5 | 6 | ... |
|----------------|---|---|---|----|---|---|---|-----|
| $7^x \bmod 15$ | 1 | 7 | 4 | 13 | 1 | 7 | 4 | ... |

La secuencia 1, 7, 4, 13, 1, 7, 4, 13, ... **se repite cada 4 pasos**  $\Rightarrow$  período  $r = 4$ .

El registro de trabajo, tras la exponenciación modular, queda **entrelazado** con el de conteo: para cada  $x$ , almacena  $7^x \bmod 15 \in \{1, 7, 4, 13\}$ .

Esta periodicidad es la “señal” que la  $\text{QFT}^{-1}$  va a convertir en picos en el dominio transformado.

## Caso $k = 7$ — el circuito concreto ( $t = 8$ , $n = 4$ )



- **Conteo:** 8 qubits ( $2^8 = 256 \geq 225 = 15^2$ ).
- **Trabajo:** 4 qubits, init  $|1\rangle = |0001\rangle$ .
- **Bloque  $U$ :** multiplicación por 7 módulo 15,  $2^j$  veces controlada por el qubit  $j$  de conteo (mostrado como caja abstracta).
- $\text{QFT}^{-1}$  sobre los 8 qubits de conteo, luego medición.

## Caso $k = 7$ — mediciones y fracciones continuas

Con  $r = 4$  y  $2^t = 256$ , los picos de la  $\text{QFT}^{-1}$  están en  $c \approx s \cdot 64$  ( $s = 0, 1, 2, 3$ ):

| $c$ | $c/256$ | fracción reducida | denominador | resultado                                                         |
|-----|---------|-------------------|-------------|-------------------------------------------------------------------|
| 0   | 0       | 0                 | —           | inútil ( $s = 0$ ), repetir                                       |
| 64  | 1/4     | 1/4               | 4           | $r = 4$ ✓                                                         |
| 128 | 1/2     | 1/2               | 2           | candidato $r = 2$ ; $7^2 \bmod 15 = 4 \neq 1 \rightarrow$ repetir |
| 192 | 3/4     | 3/4               | 4           | $r = 4$ ✓                                                         |

Con pocas mediciones, se obtiene  $r = 4$  con altísima probabilidad.

## Caso $k = 7$ — cierre del ejemplo

$N = 15 \rightarrow$  filtros OK  $\rightarrow k = 7 \rightarrow \gcd(7, 15) = 1$



[CUÁNTICO] período  $r = 4$  (medición  $64/256 \rightarrow 1/4$ )



$r$  par  $\checkmark \rightarrow p = 7^2 \bmod 15 = 49 \bmod 15 = 4$

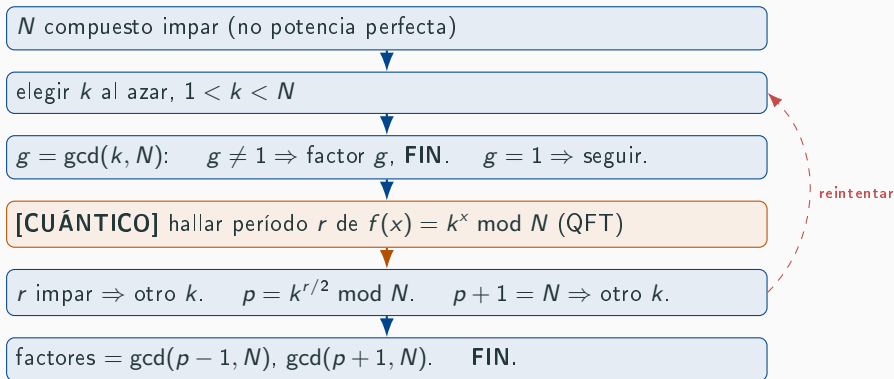


$p + 1 = 5 \neq 15 \checkmark$  (no es el modo de fallo)



$\gcd(3, 15) = 3 \quad \gcd(5, 15) = 5 \Rightarrow 15 = 3 \times 5 \checkmark \checkmark$

## El flujo completo, de un vistazo



## Complejidad e implicancias

---

# Complejidad: Shor vs. el mejor clásico (GNFS)

Comparamos **tiempo** para factorizar un entero de  $n = \log_2 N$  bits.

**GNFS (mejor clásico) — sub-exponencial**

$$\exp\left(\left(\frac{64}{9}\right)^{1/3}(\ln N)^{1/3}(\ln \ln N)^{2/3}\right)$$

Crece más lento que exponencial puro, pero **mucho** más rápido que cualquier polinomio.

**Shor (cuántico) — polinómico**

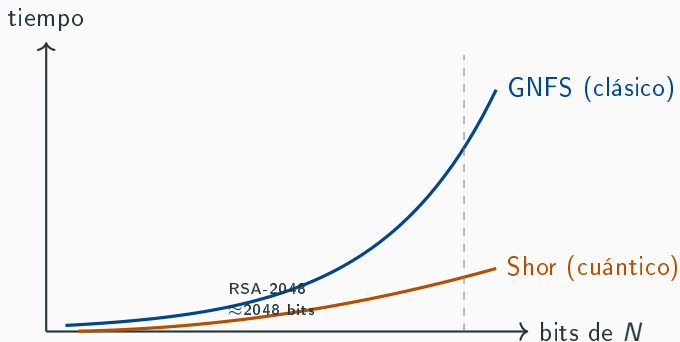
$$O((\log N)^3)$$

(versión estándar)

+ post-procesamiento clásico polinómico.

Salto cualitativo: **sub-exponencial**  $\rightarrow$  **polinómico**. No es “una constante mejor”: es otra clase de complejidad.

## Crecimiento comparativo (intuición)



Para claves RSA reales (2048+ bits), GNFS se va a tiempos astronómicos; Shor se mantiene tratable (en una máquina cuántica ideal suficientemente grande).

# Implicancias en la criptografía actual

## Qué rompe Shor

- **RSA** (factorización) — cifrado y firma.
- **DH, ElGamal, DSA** (log. discreto) — variante de Shor.
- **ECDH, ECDSA** (log. discreto en curvas) — también caen, con menos qubits que RSA.

## Qué NO rompe (o debilita levemente)

- **AES**: el ataque cuántico relevante es **Grover** (cuadrático). Se mitiga duplicando el tamaño de clave.
- **SHA-2/3**: igual, Grover cuadrático → usar hashes más largos.

La criptografía de **clave pública clásica** es la víctima principal; la simétrica sobrevive con claves más largas.

## Estado actual del hardware cuántico (2025–2026)

- **Google “Willow” (fines 2024):** 105 qubits superconductores. Demostró **corrección de errores “por debajo del umbral”** (el error lógico baja al agregar qubits físicos). Hito cualitativo, no una máquina factorizadora.
- **IBM — hoja de ruta a tolerancia a fallos:** sistema **“Starling” para 2029** con  $\sim 200$  qubits lógicos y 100M de operaciones corregidas; hitos intermedios *Loon* (2025), *Kookaburra* (2026), *Cockatoo* (2027). Meta de  $\sim 1000$  qubits lógicos a inicios de 2030, con códigos qLDPC.
- **Realidad para Shor:** no existe hoy una máquina capaz de correr Shor sobre números criptográficamente relevantes. Falta tolerancia a fallos a escala.

**Qubits físicos** (ruidosos, abundantes) vs. **qubits lógicos** (corregidos, escasos). Shor a escala RSA necesita miles de lógicos.

# El “gap” honesto: lo más grande factorizado de verdad

## En hardware cuántico real (Shor genuino)

- **15** (2001, IBM,  $\sim 8$  qubits) — récord confiable.
- **21** (2012,  $\sim 10$  qubits).
- Muchos “récores” mediáticos usan circuitos **pre-compilados** (con los factores conocidos de antemano) o métodos adiabáticos/de optimización que **no escalan**. No cuentan como Shor genuino.

## En simulación clásica

Se han simulado semiprimos de  $\sim 12$  dígitos, pero eso mide la simulación, no una ventaja cuántica real.

La distancia entre “15 en hardware” y “2048 bits en RSA” es **enorme**. La amenaza es futura y creíble, no presente.

# Estimaciones de recursos para romper RSA-2048

## Gidney & Ekerå (2019)

~20 millones de qubits físicos ruidosos, ~8 horas.

## Gidney (mayo 2025, arXiv:2505.15917)

Menos de 1 millón de qubits físicos, <1 semana.

(≈1399 qubits lógicos)

Reducción de ~20× respecto de 2019, por mejor aritmética modular y codificación más eficiente.

## Lectura

- La barrera **sigue siendo enorme** frente a las máquinas actuales.
- Las estimaciones **bajan año a año**: el blanco se acerca por mejoras de algoritmo y de corrección.
- No hace falta esperar la máquina para preocuparse.

# Criptografía post-cuántica: NIST PQC

Algoritmos diseñados para resistir a Shor y Grover, corribles en hardware clásico. **NIST** finalizó los primeros estándares:

| Estándar           | Nombre final              | Tipo              | Fecha             |
|--------------------|---------------------------|-------------------|-------------------|
| FIPS 203           | <b>ML-KEM</b> (Kyber)     | KEM (encapsulado) | ago-2024          |
| FIPS 204           | <b>ML-DSA</b> (Dilithium) | Firma digital     | ago-2024          |
| FIPS 205           | <b>SLH-DSA</b> (SPHINCS+) | Firma (hash)      | ago-2024          |
| FIPS 206           | <b>FN-DSA</b> (Falcon)    | Firma digital     | en proceso (2026) |
| (5. <sup>o</sup> ) | <b>HQC</b>                | KEM (códigos)     | selec. mar-2025   |

- ML-KEM, ML-DSA, FN-DSA: basados en **retículos** (lattices).
- SLH-DSA: basado en **hashes**.
- HQC: basado en **códigos** (diversificación frente a retículos).

## Qué hacer hoy: “harvest now, decrypt later”

- **Amenaza presente aunque la máquina sea futura:** un adversario puede **capturar hoy** tráfico cifrado y **descifrarlo mañana** cuando exista la computadora cuántica. Datos con larga vida útil (secretos de estado, salud, propiedad intelectual) ya están en riesgo.
- **Por eso la migración empezó:** estándares finalizados (2024–2025); TLS ya incorpora **modos híbridos** (clásico + ML-KEM).
- **Recomendación práctica:** inventariar dónde se usa clave pública, priorizar lo de larga vida, y adoptar esquemas **híbridos** durante la transición.

### Híbrido

Se combinan un esquema clásico y uno PQC; el atacante tiene que romper **ambos**. Da seguridad durante la incertidumbre de transición.

## Conclusiones y referencias

---

# Conclusiones

- Shor **reduce factorizar a hallar el período** de  $f(x) = k^x \bmod N$ ; ese período es lo **único cuántico**.
- Cadena algebraica cerrada:  $r$  par  $\rightarrow (k^{r/2} - 1)(k^{r/2} + 1) \rightarrow$  evitar  $k^{r/2} \equiv -1 \rightarrow \gcd(k^{r/2} \pm 1, N)$  da factores, con **éxito  $\geq 1/2$  por intento**.
- La **QFT** extrae el período; es exponencialmente más barata en operaciones, pero solo permite **muestrear** la salida.
- Complejidad: **polinómica** (Shor) vs. **sub-exponencial** (GNFS) — un salto de clase.
- **Hoy** no hay hardware para romper RSA real (récores: 15 y 21), pero las hojas de ruta y estimaciones decrecientes hacen la amenaza **creíble**.
- La respuesta existe: **NIST PQC** (ML-KEM, ML-DSA, SLH-DSA, + HQC) y migración **híbrida**, motivada por *“harvest now, decrypt later”*.

## Estándar técnico:

- M. Nielsen, I. Chuang, *Quantum Computation and Quantum Information*, Cambridge — cap. 5, Teorema A4.13.
- P. Shor, “Algorithms for Quantum Computation”, FOCS 1994.

## Material complementario:

- C. H. Ugwuishiwu et al., “An Overview of Quantum Cryptography and Shor’s Algorithm”, *IJATCSE* 9(5), 2020.
- Clase 04 — Criptografía asimétrica, ITBA.

## Hardware y estimaciones:

- Google Quantum AI, chip Willow, 2024–2025.
- IBM, hoja de ruta Starling 2029, jun-2025.
- C. Gidney, arXiv:2505.15917 (2025)  
— <1M qubits físicos para RSA-2048.
- C. Gidney, M. Ekerå (2019)  
— 20M qubits, 8h.

## NIST PQC:

- FIPS 203/204/205 — 13-ago-2024.
- HQC seleccionado — 11-mar-2025.